UCS503 - Software Engineering

Campus Marketplace

A Verified Student Platform for Buying, Selling, Lending and Borrowing with AI-Driven Pricing and Fraud Detection

Submitted by:

1024031049	Shrasthi Singh Rathore
1024031021	Raksha Pandey
1024031047	Shweta Agarwal

Group: 3C73, BE Third Year, CSE

Submitted to:

Dr. Jhonsy Bansal

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**
1.1 Background and Context 2
1.1.1 Problem Statement 2
1.2 Project Objectives 2

2 Methodology and Design **3**
2.1 System Architecture 3
2.1.1 High-Level Design 3
2.1.2 Technical Stack 3
2.2 Implementation Details 4
2.2.1 Module 1: Core Functionality 4
2.2.2 Module 2: Secondary Features 4

3 Results and Evaluation **6**
3.1 Testing Methodology 6
3.2 Performance Metrics 6
3.3 Analysis 6

4 Conclusion **8**
4.1 Summary of Work 8
4.2 Key Findings 8
4.3 Future Work and Recommendations 8
4.4 Final Remarks 9

Chapter 1

Introduction

1.1 Background and Context

College campuses generate a constant churn of used books, hostel items, cycles, electronics and lecture notes that students want to sell, lend, buy or borrow, but there is no single trusted place to do this. Existing general-purpose classifieds and social-media groups have no way of verifying that a seller or buyer is actually a student of the same institute, no structured record of listings, messages or transactions, and no way to judge whether a quoted price is fair or whether a listing looks fraudulent. **Campus Marketplace** is a full-stack web application built to solve this problem: a campus-exclusive marketplace where every user is verified through their institutional email, and where an integrated machine-learning layer assists with fair pricing and fraud screening.

1.1.1 Problem Statement

Students need a single, trustworthy platform, restricted to verified members of their own institute, on which they can browse and publish listings for buying, selling, lending and borrowing items; exchange messages with other users; track transactions and due dates for borrowed items; and rely on the platform to flag unfair prices and suspicious listings, all of this backed by persistent storage rather than data that disappears on refresh.

1.2 Project Objectives

1. **Objective 1:** Build a fully functional MERN-stack backend (Express + MongoDB via Mongoose) so that listings, users, messages, transactions, reviews and moderation cases are persisted in a real database instead of hardcoded mock data.
2. **Objective 2:** Restrict access to verified students by implementing institutional email verification during sign-up.
3. **Objective 3:** Support the full buy/sell/lend/borrow lifecycle, including due-date tracking for items that are lent or borrowed rather than sold outright.
4. **Objective 4:** Integrate machine-learning services for AI-based price recommendation and fraud risk scoring so that listings are priced fairly and suspicious activity is flagged automatically.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The application follows a modular, three-part architecture. A React (Vite) single-page frontend handles routing, filtering and all user-facing screens (Buy/Borrow, Sell/Lend, My Listings, Messages, Transactions, Profile and an Admin moderation view). An Express.js REST API sits between the frontend and the database, exposing endpoints for listings, users, messages, transactions, reviews and moderation, and enforcing business rules such as seller identity and category metadata on new listings. MongoDB, accessed through Mongoose models, is the persistent data store; during development, when a local MongoDB instance is not available, the backend automatically falls back to an in-memory MongoDB instance so the system remains fully runnable and demoable. Alongside this Node/Express core, independent machine-learning microservices handle price recommendation and fraud scoring.

2.1.2 Technical Stack

- **Frontend:** React (Vite), with Lucide React icons.
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB (Mongoose ODM), with `mongodb-memory-server` as a development-time fallback, and MongoDB Atlas for hosted deployment.
- **Machine Learning Services:** Python, FastAPI, scikit-learn (Random Forest / XGBoost for price recommendation, Isolation Forest for fraud risk scoring).
- **Tools:** `cors` for cross-origin access, Vite dev-server proxying, npm scripts for running and seeding the backend.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core of this phase of the project was converting a frontend-only prototype into a fully functional client-server application:

- A dedicated `server/` package was created with Express and Mongoose, defining `User`, `Listing`, `Message`, `Transaction`, `Review` and `Moderation` models, and exposing REST routes under `/api/...`.
- Every major frontend page was wired to the API through a centralised `src/api.js` module, so that Buy/Borrow, Sell/Lend, My Listings, Messages, Transactions, Profile and Admin all read and write MongoDB instead of hardcoded arrays.
- Publishing a listing now creates a real document in MongoDB (`POST /api/listings`) with seller identity and category/fair-price metadata attached, and My Listings, Buy/Borrow and the Dashboard all refetch from the database, so new listings persist across page refreshes.
- "Chat with seller" and "Buy/Borrow" actions, which previously only showed a confirmation toast, now create real `Message` and `Transaction` documents, so conversation and purchase history accumulates with real use.
- Borrow/lend transactions carry a due date, and the application tracks and surfaces this due date so borrowed items can be returned on time and lenders can see when an item is expected back.
- CORS was enabled on the Express server and a Vite dev-server proxy was configured so the frontend (port 5173) can call `/api/...` and have it forwarded transparently to the API (port 5001), eliminating cross-origin and wrong-origin errors during development.
- Because Lucide icon components cannot be stored in MongoDB, listings store a string `iconKey` field (e.g. `"books"`, `"electronics"`); a `hydrateListing()` helper on the frontend maps this key back to the correct icon component before rendering.
- A seed script (`server/seed.js`) populates the database with realistic sample data (15 listings across books, cycles, hostel items, notes and electronics, along with users, message threads, transactions, reviews and moderation cases) automatically on first run when the listings collection is empty, so the UI is demo-ready even before real usage data accumulates.

2.2.2 Module 2: Secondary Features

On top of the core CRUD backend, the following supporting features have been completed:

- **Institutional Email Verification:** Sign-up is restricted to users with a valid institutional email address, so that only verified students of the campus can list, message or transact on the platform.

- **AI Price Recommendation:** A FastAPI microservice hosts a regression model (Random Forest, with an XGBoost variant) trained to suggest a fair price band for a listing based on category, condition and comparable items, helping sellers price items reasonably and buyers judge whether an asking price is fair.
- **Fraud Detection:** A companion FastAPI microservice runs an Isolation Forest model to assign a fraud risk score to listings and transactions, flagging anomalous pricing or behaviour patterns for admin review through the moderation queue.
- **Borrow/Lend Due-Date Tracking:** Transactions created for lend/borrow listings (as opposed to outright sales) carry a due date, allowing both parties to track when a borrowed item is due for return.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Manual/Integration Testing:** Each API route was exercised end-to-end from the corresponding frontend page (e.g. publishing a listing from Sell/Lend and confirming it appears under My Listings and Buy/Borrow after a refresh).
- **Fallback Testing:** The in-memory MongoDB fallback was verified by running the server without a local `mongod` instance and confirming the API still started, connected and served data correctly.
- **Seed Verification:** The auto-seed logic was verified by starting against an empty database and confirming that listings, users, messages, transactions, reviews and moderation cases were created automatically.

3.2 Performance Metrics

The following was verified during a live run of the system:

Item	Result
API server	Running on port 5001, connected in memory-DB mode
Seeded listings	15 (books, cycles, hostel items, notes, electronics)
Seeded message threads	3
Seeded transactions	3
Seeded reviews	3
Seeded moderation cases	3
Frontend data source	Loaded live from MongoDB via React UI

Table 3.1: Verified system state after a clean start and auto-seed

3.3 Analysis

All four project objectives set out in Section 1.2 were met for this phase: a working persistent backend was built, institutional email verification restricts access to verified stu-

dents, the full buy/sell/lend/borrow lifecycle (including due-date tracking) is supported, and both the AI price-recommendation and fraud-detection services are integrated. The principal implementation challenges, and how each was resolved, are summarised below.

- **No backend existed initially** – resolved by building a dedicated Express + Mongoose `server/` package and wiring every frontend page to it through a single `api.js` module.
- **MongoDB was not installed on the development machine** – resolved by adding an automatic `mongodb-memory-server` fallback, so the app runs identically whether or not a local database is available, while keeping a real `MONGODB_URI` ready for production.
- **Frontend icon components could not be persisted** – resolved by storing a string `iconKey` and hydrating it back to a Lucide icon on the client.
- **CORS/port mismatch between Vite and Express** – resolved with `cors()` on the server and a Vite proxy for `/api/...`.
- **Empty database looked broken in demos** – resolved with an auto-seed script that populates realistic sample data on first run.
- **Publishing/chat/buy actions did not persist** – resolved by backing each action with a real MongoDB document (`Listing`, `Message`, `Transaction` respectively).

Chapter 4

Conclusion

4.1 Summary of Work

This phase converted Campus Marketplace from a frontend-only, mock-data prototype into a fully functional MERN application with a persistent MongoDB backend, institutional email verification, complete buy/sell/lend/borrow support with due-date tracking for borrowed items, and two integrated machine-learning services for AI-based price recommendation and fraud detection.

4.2 Key Findings

- An in-memory MongoDB fallback is an effective way to keep a database-backed application fully runnable during development and demos when a local database installation is unavailable.
- Splitting concerns cleanly – API layer, Mongoose models, seed script and frontend hydration – made it straightforward to replace mock data with real persistence without a full frontend rewrite.
- Keeping the ML services (pricing and fraud detection) as independent FastAPI microservices, separate from the Node/Express core, kept the system modular and easier to test in isolation.

4.3 Future Work and Recommendations

The following enhancements are identified as valuable extensions beyond the current MVP:

1. A dedicated mobile application.
2. Computer-vision-based fraud detection (e.g. verifying listing photos).
3. OCR-based invoice/receipt verification for transactions.
4. An AI chat assistant to help users navigate listings.
5. In-app escrow and payment handling.

6. Demand forecasting to guide sellers on what to list and when.
7. Opt-in geospatial discovery of nearby listings using MongoDB's 2dsphere indexing.

4.4 Final Remarks

The hardest part of this phase was not the interface, which already existed, but making the backend genuinely work end-to-end – persisting data reliably even without a local MongoDB installation, and turning previously stubbed actions (chat, buy, publish) into real, persistent records. With the database, verification, due-date tracking, pricing model and fraud-detection service all now in place, the platform is ready for the next phase of testing and refinement.